Checklista: zanim wypuścisz apkę z AI

Bartek Filipiuk · security.devince.dev · wersja z listopada 2026 ·

Wstęp

12 rzeczy, które agent AI najczęściej zostawia otwarte w apce, i jak każdą sprawdzić w kilka minut. Każdy punkt to jeden konkretny zysk: mniej wycieków, mniejszy rachunek, mniej nerwów po starcie. Nie musisz znać się na bezpieczeństwie. Potrzebujesz repo i terminala albo swojego agenta.

Zacznij od promptu na dole strony. Agent przejdzie Twoją apkę tylko czytając kod i zrobi mapę, od której łatwiej przejść przez listę.

1 Nikt nie wyda pieniędzy z Twojego konta AI ani płatności

Dlaczego. Zmienna z prefiksem `NEXT_PUBLIC_`, `VITE_`, `EXPO_PUBLIC_` albo `REACT_APP_` trafia do kodu w przeglądarce. Jeśli siedzi w niej klucz do API modelu, Stripe albo Supabase z uprawnieniami serwera, każdy może go skopiować.

Sprawdź.

```
grep -rnE "(NEXT_PUBLIC|VITE|EXPO_PUBLIC|REACT_APP)_[A-Z_]*
(SECRET|KEY|TOKEN)" --include="*.env*" --include="*.ts" --
include="*.tsx" --include="*.js" . | grep -v node_modules
grep -rniE "service_role" --include="*.ts" --include="*.tsx" --
include="*.js" --include="*.env*" . | grep -v node_modules
```

Problem wygląda tak. `NEXT_PUBLIC_OPENAI_API_KEY`, `VITE_STRIPE_SECRET_KEY` albo klucz `service_role` w pliku, który trafia do przeglądarki (komponent z `"use client"`, kod frontendu). Ten sam klucz w kodzie serwera albo w funkcji Edge jest w

porządku. Klucz publiczny z założenia (Supabase anon, klucz publikowalny Stripe, analityka) jest w porządku.

2 Użytkownik widzi tylko swoje dane w bazie

Dlaczego. W Supabase i Firebase przeglądarka rozmawia z bazą bezpośrednio. Bez reguł dostępu każdy zalogowany, a czasem każdy w ogóle, czyta całą tabelę. Tak wyciekły dane w 170 apkach z Lovable (Matt Palmer, 29.05.2025).

Sprawdź. W Supabase, w SQL Editor:

```
select tablename, rowsecurity from pg_tables where schemaname =
'public';
```

W Firebase przejrzyj `firestore.rules` i `storage.rules`.

Problem wygląda tak. `rowsecurity = false` przy tabeli z danymi użytkowników. Polityka z `using (true)` na tabeli z cudzymi danymi. W Firebase `allow read, write: if true` albo `if request.auth != null` bez sprawdzenia, czyj to dokument.

3 Zmiana numeru w adresie nie pokazuje cudzego konta

Dlaczego. Agent chętnie pisze „pobierz fakturę po id”. Jeśli kod nie sprawdza, czyja to faktura, wystarczy zmienić liczbę w adresie.

Sprawdź. Załóż dwa konta testowe. Na pierwszym otwórz dowolny swój rekord i skopiuj adres. Wklej go na drugim koncie.

Problem wygląda tak. Drugie konto widzi rekord pierwszego. W kodzie zapytanie filtruje tylko po `id`, bez warunku na właściciela albo organizację.

4 Panel admina otwiera tylko admin

Dlaczego. Ukrycie linku w menu to nie zabezpieczenie. Liczy się sprawdzenie roli po stronie serwera, w każdej trasie admina i każdym endpoincie `/api/admin`.

Sprawdź. Zaloguj się jako zwykły użytkownik i otwórz adresy panelu admina oraz jego endpointy. Poproś agenta: „Pokaż plik i linię, w której sprawdzasz rolę dla każdej trasy admina”.

Problem wygląda tak. Strona albo odpowiedź API ładuje się zwykłemu użytkownikowi. Agent nie umie wskazać linii ze sprawdzeniem roli.

5 Nikt nie da sobie planu premium fałszywym webhookiem

Dlaczego. Endpoint webhooka płatności jest publiczny. Jeśli nie sprawdza podpisu, każdy może wysłać „płatność zakończona” i dostać płatny dostęp za darmo.

Sprawdź.

```
grep -rlE
"webhook|checkout\.session\.completed|payment_intent\.succeeded"
--include="*.ts" --include="*.js" --include="*.py" --
include="*.php" . | grep -v node_modules
```

W znalezionym pliku szukaj `constructEvent` (Stripe w Node), `construct_event` (Python) albo innego sprawdzenia podpisu przed odczytem treści.

Problem wygląda tak. Handler robi `await request.json()` i od razu nadaje plan albo zapisuje zamówienie.

6 Rachunek za API nie wystrzeli przez jednego użytkownika

Dlaczego. SMS, maile i wywołania modelu kosztują przy każdym wywołaniu. Endpoint bez limitu to otwarty portfel.

Sprawdź. Poproś agenta o listę endpointów, które wysyłają SMS lub mail albo wołają model AI, i o miejsce, gdzie jest limit na użytkownika lub adres IP. Sprawdź też limit wydatków w panelu dostawcy modelu i bramki SMS.

Problem wygląda tak. Brak limitu w kodzie. Użytkownik sam wybiera model albo liczbę tokenów. W panelu dostawcy brak limitu miesięcznego.

7 Plik od użytkownika nie nadpisze cudzego i nie zostanie stroną

Dlaczego. Upload bez kontroli typu, rozmiaru i ścieżki pozwala wrzucić HTML albo nadpisać plik innej osoby.

Sprawdź. Znajdź endpoint uploadu i sprawdź, kto wybiera nazwę i ścieżkę pliku, jakie typy są dozwolone i jaki jest limit rozmiaru. W Supabase Storage sprawdź, czy bucket z plikami użytkowników nie jest publiczny.

Problem wygląda tak. Ścieżka albo klucz pliku przychodzi z formularza. Brak sprawdzenia typu. Publiczny bucket z prywatnymi dokumentami.

8 Stare hasło nie wisi w historii repo

Dlaczego. Usunięcie pliku `.env` w nowym commicie nie usuwa go z historii. Jeśli repo jest albo będzie publiczne, klucz da się wyciągnąć.

Sprawdź.

```
git log --all --oneline -- .env .env.local .env.production
```

Dokładniej: `gitleaks detect` (narzędzie open source) na całym repo.

Problem wygląda tak. Jakikolwiek wynik pierwszej komendy. Wtedy zmień klucze u dostawców, samo czyszczenie historii nie wystarczy.

9 Biblioteki w apce nie mają znanych dziur

Dlaczego. Agent instaluje to, co zna, często w starszej wersji. Znane podatności w zależnościach to najtańsza do naprawienia rzecz na tej liście.

Sprawdź. Zależnie od stacku:

```
npm audit --omit=dev
pnpm audit --prod
pip-audit
composer audit
```

Problem wygląda tak. Wpisy `high` albo `critical` w paczkach, które trafiają na produkcję.

10 Treść od użytkownika i od modelu AI nie odpala skryptu

Dlaczego. Tekst wstawiony jako HTML może uruchomić kod u innego użytkownika. Dotyczy to też odpowiedzi modelu, jeśli model widział treść od użytkowników.

Sprawdź.

```
grep -rnE "dangerouslySetInnerHTML|v-html|innerHTML|\
{!!|\|safe|\|raw" --include="*.tsx" --include="*.jsx" --
include="*.vue" --include="*.js" --include="*.html" --
include="*.php" --include="*.twig" . | grep -v node_modules
```

Problem wygląda tak. Wstawianie jako HTML treści, którą ktoś może wpisać: komentarz, opis, notatka, odpowiedź modelu. Bez czyszczenia biblioteką typu DOMPurify.

11 Usunięcie konta naprawdę usuwa dane

Dlaczego. Użytkownik ma prawo do usunięcia danych. Usunięcie wiersza w bazie często zostawia pliki w storage i rekordy w innych tabelach.

Sprawdź. Poproś agenta: „Pokaż, co dokładnie usuwa funkcja usuwania konta: tabele, pliki, wpisy w zewnętrznych usługach”.

Problem wygląda tak. Pliki użytkownika zostają w buckecie. Dane zostają w tabelach bez powiązania kaskadowego albo w usłudze mailingowej.

12 Przeglądarka pomaga Ci bronić apki

Dlaczego. Kilka nagłówków odpowiedzi blokuje całe klasy ataków bez zmiany kodu aplikacji. Zbyt szeroki CORS z ciasteczkami robi odwrotnie.

Sprawdź.

```
curl -sI https://twoja-apka.pl | grep -iE "content-security-policy|strict-transport-security|x-frame-options|x-content-type-options|access-control-allow"
```

Problem wygląda tak. Brak nagłówków `strict-transport-security` i `content-security-policy`. `access-control-allow-origin` ustawiony na adres z żądania razem z `access-control-allow-credentials: true`.

Prompt do skopiowania: mapa Twojej apki

Wklej do Claude Code, Cursora albo OpenCode w katalogu swojego repo.

```
Przejdź po tym repozytorium wyłącznie czytając pliki. Nie
uruchamiaj aplikacji, testów ani skryptów, nie wysyłaj żądań do
sieci i niczego nie zmieniaj.

Zrób mapę bezpieczeństwa mojej apki, w pięciu częściach:

1. Punkty wejścia: każda trasa strony, endpoint API, server
action, webhook i zadanie w tle. Przy każdym: kto może go wywołać
(każdy, zalogowany, admin, zewnętrzna usługa) i plik:linia, gdzie
to jest sprawdzane. Jeśli nie widzisz sprawdzenia, napisz „nie
znalazłem”.
2. Dane: gdzie apka trzyma dane użytkowników i płatności, i jak
zapytania ograniczają je do właściciela (plik:linia albo „nie
znalazłem”). Jeśli jest Supabase albo Firebase, wypisz tabele i
kolekcje oraz to, czy mają reguły dostępu.
3. Sekrety: każda zmienna środowiskowa, której używa kod, z
informacją, czy trafia do przeglądarki.
4. Kosztowne akcje: wysyłka SMS, maili i wywołania modeli AI,
każda z informacją o limicie albo „brak limitu”.
5. Webhooki i uploady: czy sprawdzają podpis, typ, rozmiar i
ścieżkę.

Zakończ listą maksymalnie pięciu miejsc, które sprawdziłbyś
najpierw, z uzasadnieniem w jednym zdaniu. Jeśli czegoś nie
jesteś pewien, napisz to wprost. Nie zgaduj i nie wymyślaj
ścieżek, których nie ma w repo.
```

Wynik to Twoja mapa. Najwięcej dają miejsca z „nie znalazłem”.

Na koniec

Ta lista to ręczna wersja tego, co skill security-audit robi automatycznie na całym kodzie, z dowodem przy każdym znalezisku. Chcesz przejść to na swojej apce z grupą, z kartą

skanu pod swój stack i z moimi odpowiedziami? Sprint bezpieczeństwa, 11–22 stycznia 2027: security.devince.dev.